Procedural Content Generation

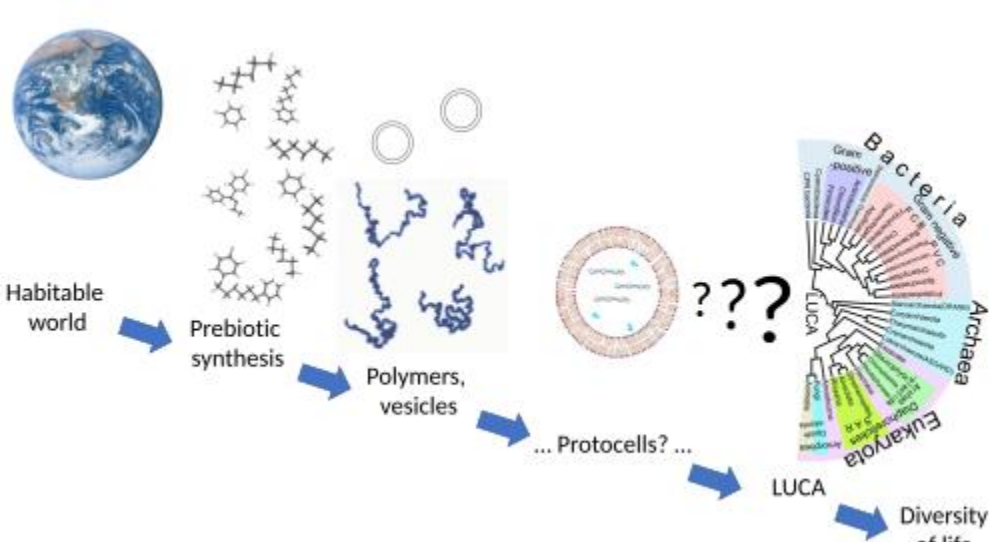
- Emergence vs Control
- Multi-scale simulation problems
- Agents
- Cellular Automata

Procedural Content Generation

- **Emergence vs Control**
- Cellular Automata
- Multi-scale simulation problems
- Agents

Emergence

Emergence refers to **complex, unexpected, or self-organizing outcomes** that arise from relatively simple rules.



Exploration

Complexity

Surprise

Emergence

Autonomy

In PCG:

- Simple algorithms generate rich structures
- Systems interact in nonlinear ways
- Designers don’t explicitly specify the final result

Minecraft – simple terrain rules → infinite landscapes.

Terrain generation plus player interaction creates structures and experiences no one directly designed.

No Man's Sky – procedural planets → astronomical variation.

Procedural planetary systems create vast, unpredictable variation.

First-Order vs **Second-Order Emergence**

Some emergence happens **inside the system.**
Some emergence happens **between the system and the player.**

First-Order Emergence

(Systemic / Structural Emergence)

Emergent complexity that arises directly from interactions between rules inside the system.

It happens when:
Simple local rules interact
No single rule encodes the final structure
The complexity is produced internally

Key Characteristics
Algorithmic
Structural
Objective (exists even without a player)
Designer → rules → output

First-Order Emergence
(Systemic / Structural Emergence)

In **Spelunky**, level layouts emerge from:
Tile grammar rules
Placement constraints
Enemy distribution systems
The final level isn't hand-authored, it emerges from rule interaction.

In **Minecraft**, terrain emerges from:
Noise functions
Biome rules
Material layering
No one directly "designed" that mountain.

Second-Order Emergence (Experiential / Behavioral Emergence)

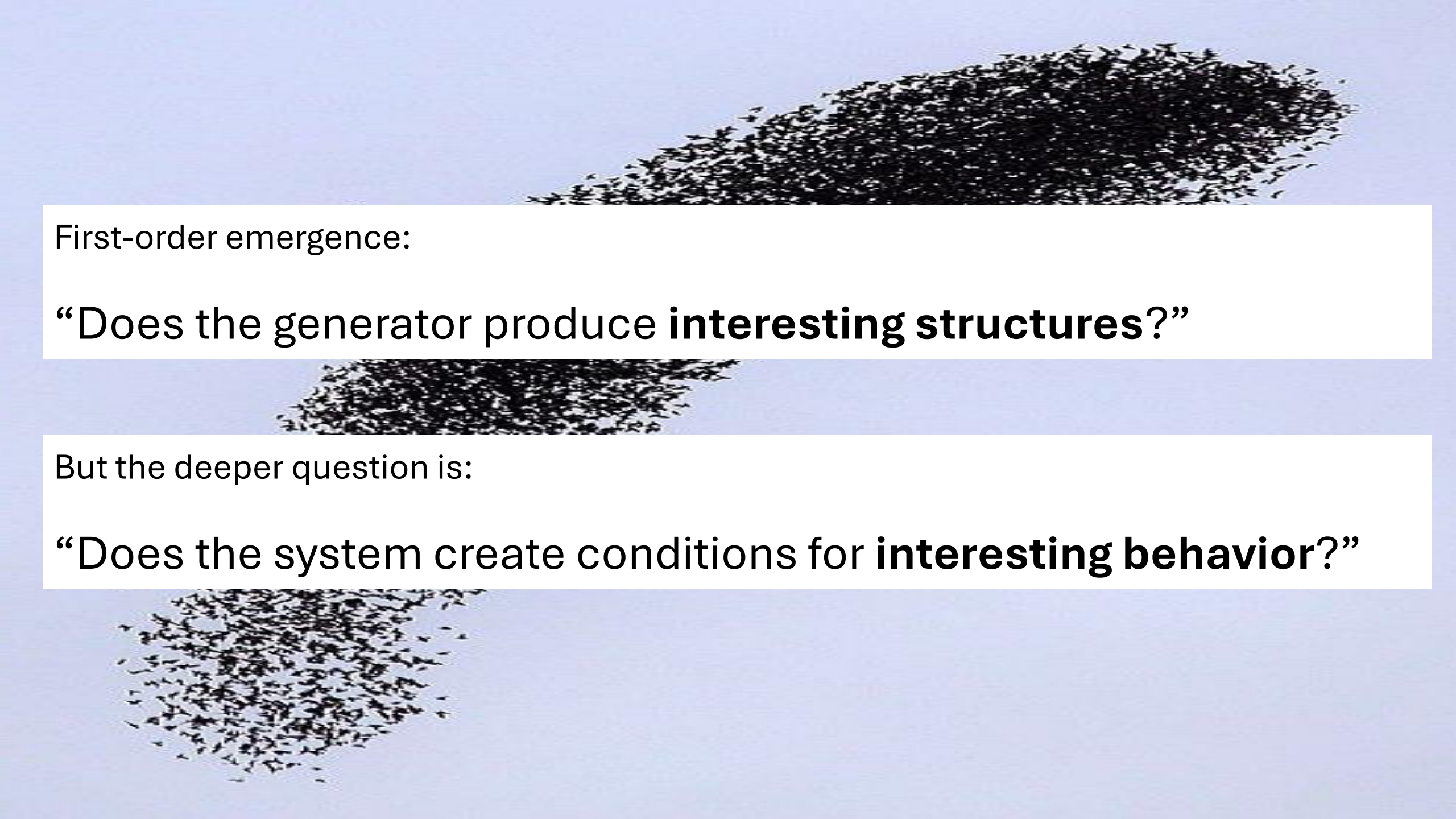
Emergence that arises from the interaction between system and player.

Key Characteristics
Relational
Experiential
Depends on players
System + human → emergent behavior

Second-Order Emergence (Experiential / Behavioral Emergence)

Left 4 Dead:
The AI Director adjusts enemy spawning (first-order system behavior).
But player panic, cooperation, betrayal, improvisation — those are second-order emergence.

Minecraft:
The terrain generation is first-order.
Players building economies, political systems, survival narratives — second-order.
Second-order emergence requires:
Interpretation
Strategy
Social dynamics
Meaning-making



First-order emergence:

“Does the generator produce **interesting structures**?”

But the deeper question is:

“Does the system create conditions for **interesting behavior**?”

The Core Tension

First-Order Emergence vs Control

Designers can **constrain rules**.
They can **validate outputs**.
They can **regulate parameters**.
This is *controllable emergence*.

Second-Order Emergence vs Control

You cannot directly control:
Player **interpretation**
Social **behavior**
Emergent **strategy**
You can only design **affordances**.

Level	Where It Happens	Example	Designer Controls
First-order	Inside the system	Terrain generation	Rules, parameters
Second-order	Between system and player	Emergent strategies	Affordances only

Control

Reliability

Intentionality

Authorial vision

Structure

Control refers to the degree to which the **designer constrains, shapes, or guarantees outcomes** in the generated content.

Quality assurance

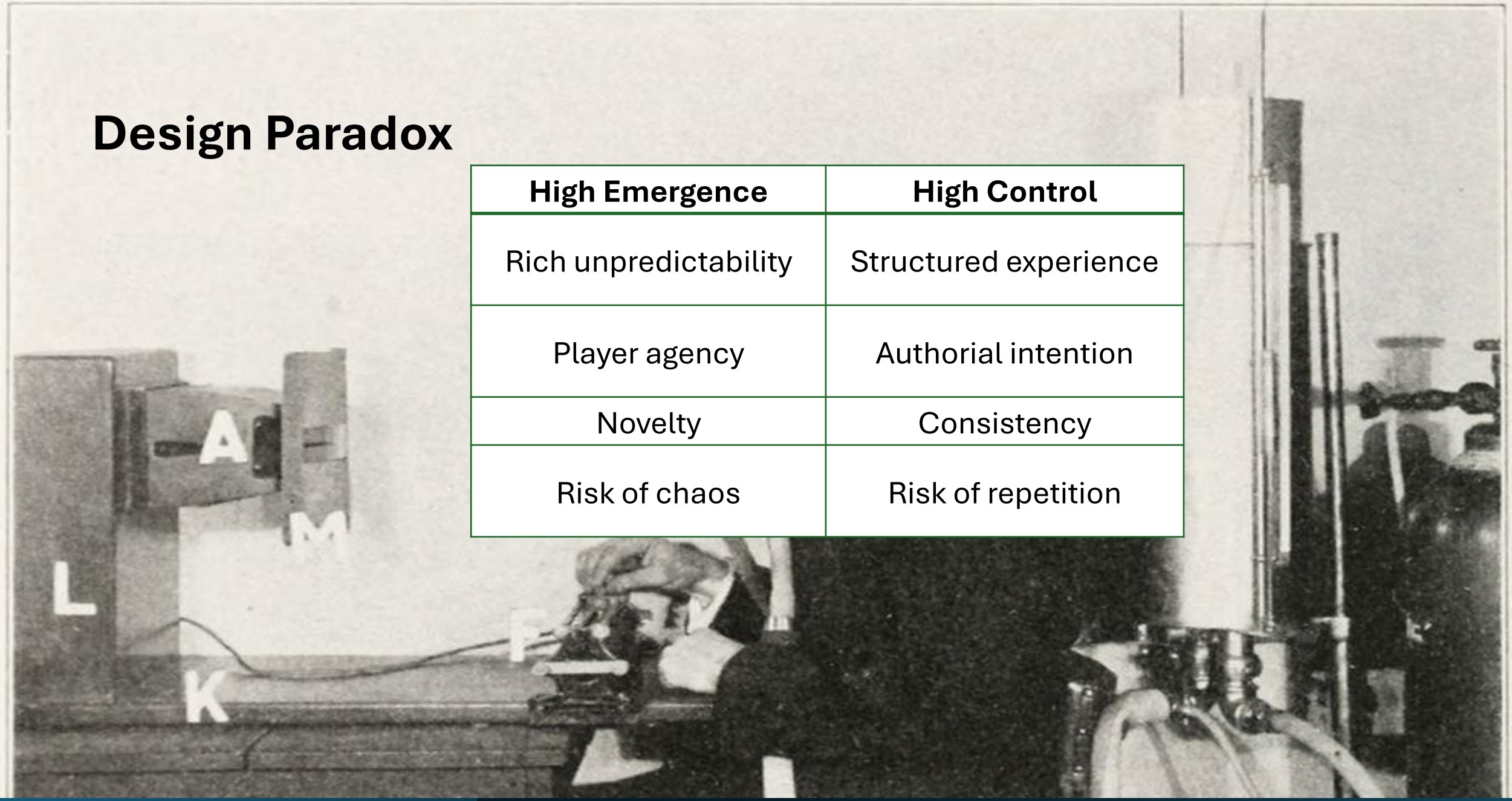
- Ensuring playability - levels are beatable
- Maintaining pacing
- Guaranteeing narrative coherence
- Avoiding broken or trivial outcomes
- Meeting aesthetic goals

Examples:

- **Spelunky** - levels are procedurally generated, but strict design rules ensure fairness and structure.
- **Left 4 Dead** – the AI Director dynamically adjusts difficulty while maintaining pacing control.

Design Paradox

High Emergence	High Control
Rich unpredictability	Structured experience
Player agency	Authorial intention
Novelty	Consistency
Risk of chaos	Risk of repetition



Why This Matters

In procedural design:

- designers are not directly creating content — they are designing **rule systems**.
- You design the *system that produces objects*.

How much autonomy do we give the system?

Too much control turns generation into automation.
Too much emergence turns design into noise.

The art of procedural design is not giving up control.
It is designing the conditions under which emergence becomes meaningful.

Are we designing outcomes, or are we designing possibility spaces?

Emergence expands the possibility space.
Control narrows it.

Good PCG doesn't eliminate one side — it balances both.

Design Strategies

1. Constrained Emergence

- Let things emerge — but inside boundaries.
- Simple systems, but within boundaries.

2. Multi-Level Control

Emergence at micro scale, control at macro scale.

- Randomized room layout
- Predefined narrative arc and overall progression fixed

3. Adaptive Systems

System monitors player behavior and adjusts.

- Dynamic difficulty
- Pacing adjustment

Complex systems theory → emergence

Cybernetics and feedback → control

Authorship vs autonomy in generative systems

Designer as meta-designer (designing systems instead of artifacts)

Procedural Content Generation

- Emergence vs Control
- **Cellular Automata**
- Multi-scale simulation problems
- Agents

Cellular Automata

- A grid of cells
- Each cell has a state
- A neighborhood definition
- A transition **rule**
- **Discrete** time steps

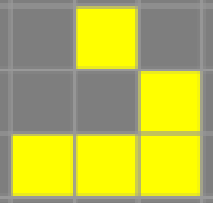
At each step:

- Every cell updates based only on its neighbors.
- No central controller.
- No global blueprint.

John Conway's Game of Life

Game of Life demonstrated:

- Complex behavior from simple rules
- Gliders
- Oscillators
- Self-replicating structures
- It proved that simple local rules can generate computation and complexity.



This insight is foundational for PCG.

https://playgameoflife.com/

Formal Definition

Let:

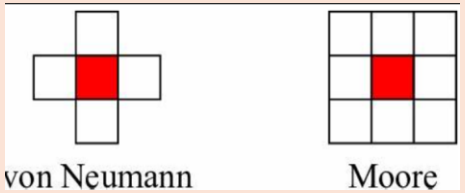
- Grid G
- State set S
- Neighborhood function N
- Transition rule f

Then:

- $S_{t+1}(x) = f\left(S_t\left(N(x)\right)\right)$

This local update rule is applied simultaneously to all cells.

Neighborhood Types



Moore Neighborhood
8 surrounding cells (square)

Von Neumann Neighborhood
4 orthogonal cells

Different neighborhoods produce different spatial aesthetics.
Moore → organic blobs
Von Neumann → more rectilinear structures

Cave Generation

Algorithm

1. Initialize grid randomly (45% walls).
2. For several iterations:
    1. If a cell has > 4 wall neighbors → becomes wall
    2. Otherwise → becomes empty

Cave - Johnson et al. Algorithm

Parameters
r = initial rock probability
N = CA iterations
T = rock threshold
Room size = 50 × 50

Algorithm Steps (Single Room)
Initialize 50×50 grid → empty cells
Sprinkle rocks → each cell becomes rock with probability r
Repeat N times:
    For each cell:
        Count rock neighbors (Moore neighborhood)
        If count ≥ T → cell becomes rock
        Else → cell becomes empty
Mark rocks adjacent to empty space as walls (visual only)

Multi-Room Generation
Generate multiple rooms independently
Identify largest empty region per room
If regions disconnected → drill corridor at closest points
Smooth edges:
Apply 2 additional CA iterations on 3×3 room block

Referencia: Johnson, L., Yannakakis, G.N., Togelius, J.: Cellular autómata for real time generation of Infinite cave levels. In Proceedings of the 2010 Workshop on Procedural Content Generation in Games (2010)

Advanced Uses in PCG

Forest Spread Simulation

Rules:

- Trees grow if surrounded by trees.
- Burn if adjacent to fire.
- Regrow probabilistically.

Generates:

- Forest patches
- Fire spread
- Regeneration cycles

Advanced Uses in PCG

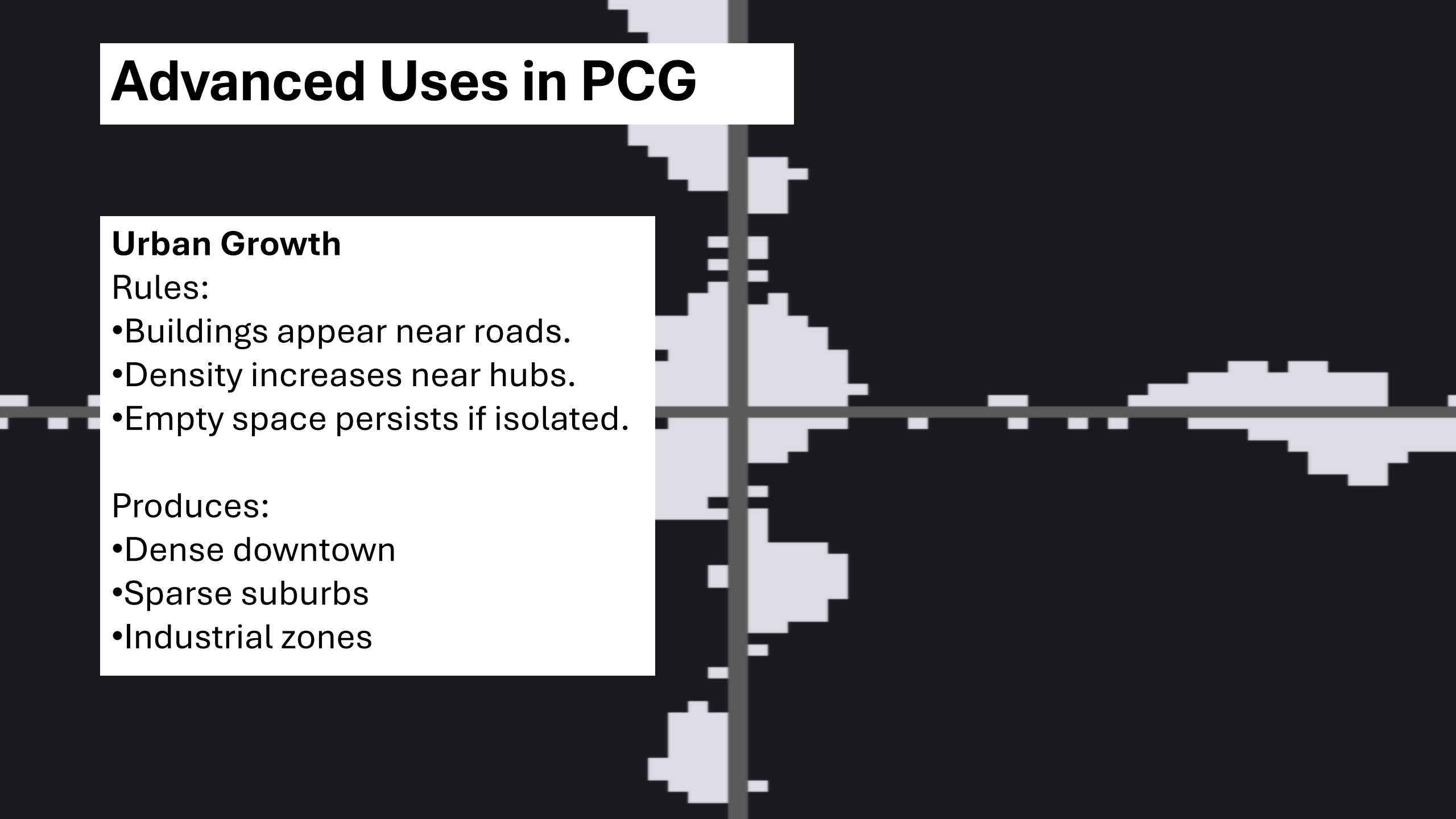
Urban Growth

Rules:

- Buildings appear near roads.
- Density increases near hubs.
- Empty space persists if isolated.

Produces:

- Dense downtown
- Sparse suburbs
- Industrial zones



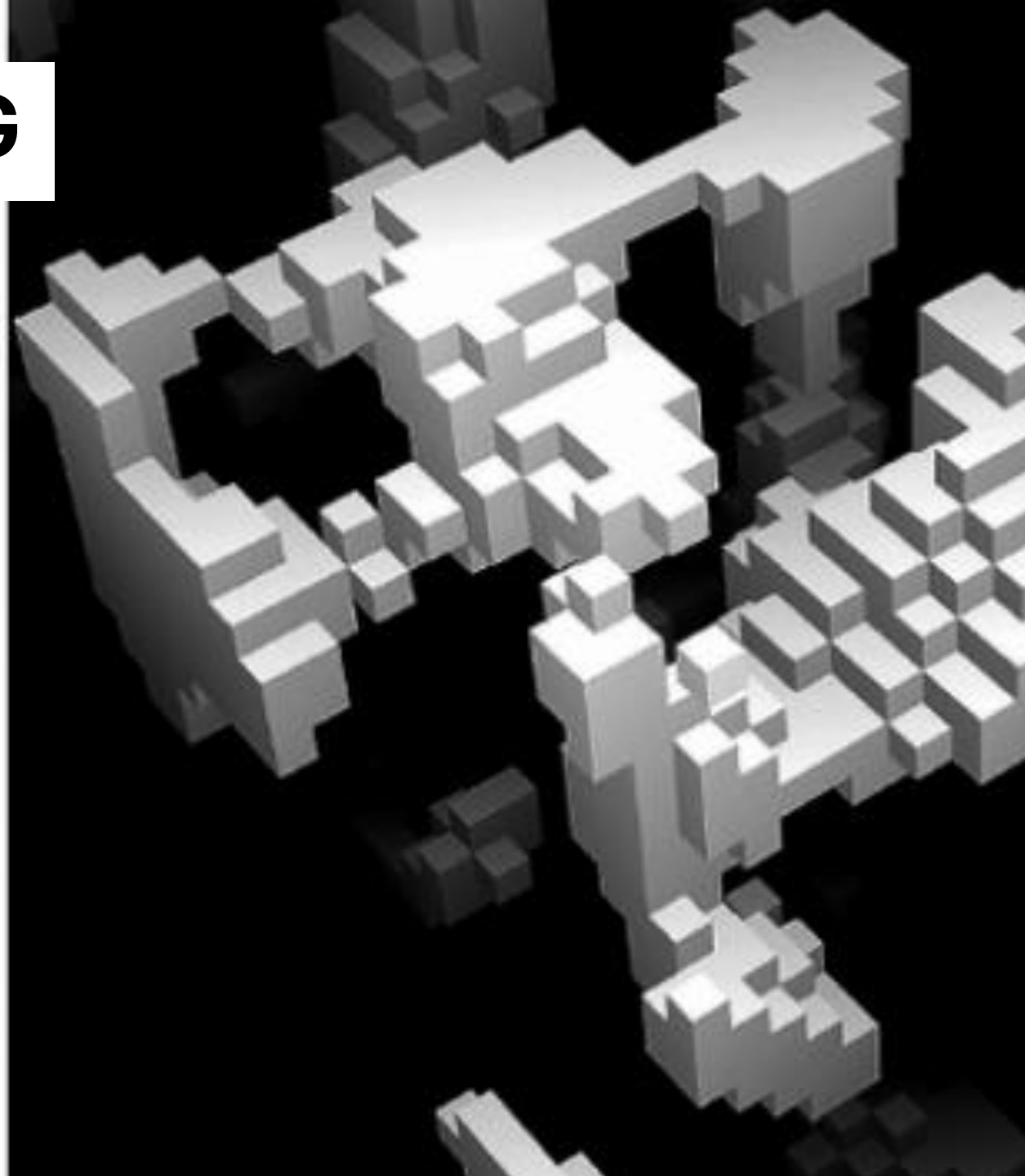
Advanced Uses in PCG

Architecture Synthesis

CA can generate:

- Facade patterns
- Window arrangements
- Decorative tiling

Often combined with grammar systems.



Why Cellular Automata?

Discrete
Rule-based
Local
Iterative
Emergent

They naturally produce:

- Organic shapes
- Spatial clustering
- Natural boundaries
- Emergent structures

And are:

- Simple to implement
- Highly parallelizable
- Easy to parameterize
- Emergent behavior

They are excellent for:

- Cave systems
- Terrain smoothing
- Dungeon layouts
- Forest growth
- Urban spread
- Pixel-art textures

Limitations

- Limited long-range planning
- Hard to enforce global constraints
- Connectivity issues
- Difficult to guarantee playability

Phase Behavior in CA

CA sit between:

- Pure randomness and Explicit design

They demonstrate:

- Global order can arise from purely local computation.
- They are models of:
- Distributed intelligence
- Decentralized generation
- Emergent geometry

CA systems can operate in:

•Ordered regime
•Chaotic regime
•Edge-of-chaos regime

CA systems exhibit:

•Stable states
•Oscillations
•Chaos
•Self-organized patterns

The most interesting PCG outputs appear **near criticality**.
This is where:
•Structure exists
•Variation persists
•Predictability coexists with surprise

In PCG terms:

•Too few iterations → noisy maps
•Too many iterations → uniform blocks
•Balanced iterations → organic design
This links directly to complexity theory.

Procedural Content Generation

- Emergence vs Control
- Cellular Automata
- **Agents**
- Multi-scale simulation problems

Agents

Conceptual Model

Environment = mutable state

Agents = forces acting on that state

Content = history of transformations

The creation of game content through the **autonomous behavior** and **interaction** of simulated entities operating across multiple scales.

Agents directly **modify the environment** over time, and the final generated content is the **accumulated result** of their actions.

The world is shaped.

This mirrors **natural processes**:

- **Ants** build colonies
- **Rivers** carve valleys
- Humans construct **cities**

The generator is no longer a function.
It is a simulation.

Pattern 1 - **Sculpting Behaviors**

Digging Agents	Building Agents	Terraforming Agents
• Tunnel creation • Dungeon carving • Mining systems	• Settlement expansion • Road formation • Architecture growth	• Erosion droplets • Lava flow simulation • Vegetation spread
Each agent: Moves through space Removes material Leaves structural traces Used in cave generation systems.	Instead of placing a city blueprint: • Agents seek resources • Expand toward water • Connect to other settlements • Densify near activity hubs Cities emerge through repeated local decisions.	Example: Thousands of water agents simulate erosion → river basins form naturally

Strengths

- Produces organic forms
- Generates believable spatial logic
- Embeds causality into geometry
- Supports emergence

Risks

- Computational cost
- Hard to control final outcome
- Can produce chaotic or degenerate structures
- Multi-scale instability

Design Patterns in Agent-Based PCG

Pattern 1 — Agents as **Sculptors**
Agents modify environment over time.

Example:
- Digging
- Building
- Terraforming

Pattern 2 — Agents as **Evaluators**
Agents judge generated content and reshape it.

Example:
AI Director adjusting difficulty.

Pattern 3 — Agents as **Story Generators**
Narrative emerges from agent goals and conflicts.

Used in:
RimWorld
Storyteller AI controls pacing but does not script events.

Agent-Based Dungeon Generation

Core Idea

- Usually uses a *single* ***digger agent***
- Micro-level procedural approach
- Emergent structure rather than global planning

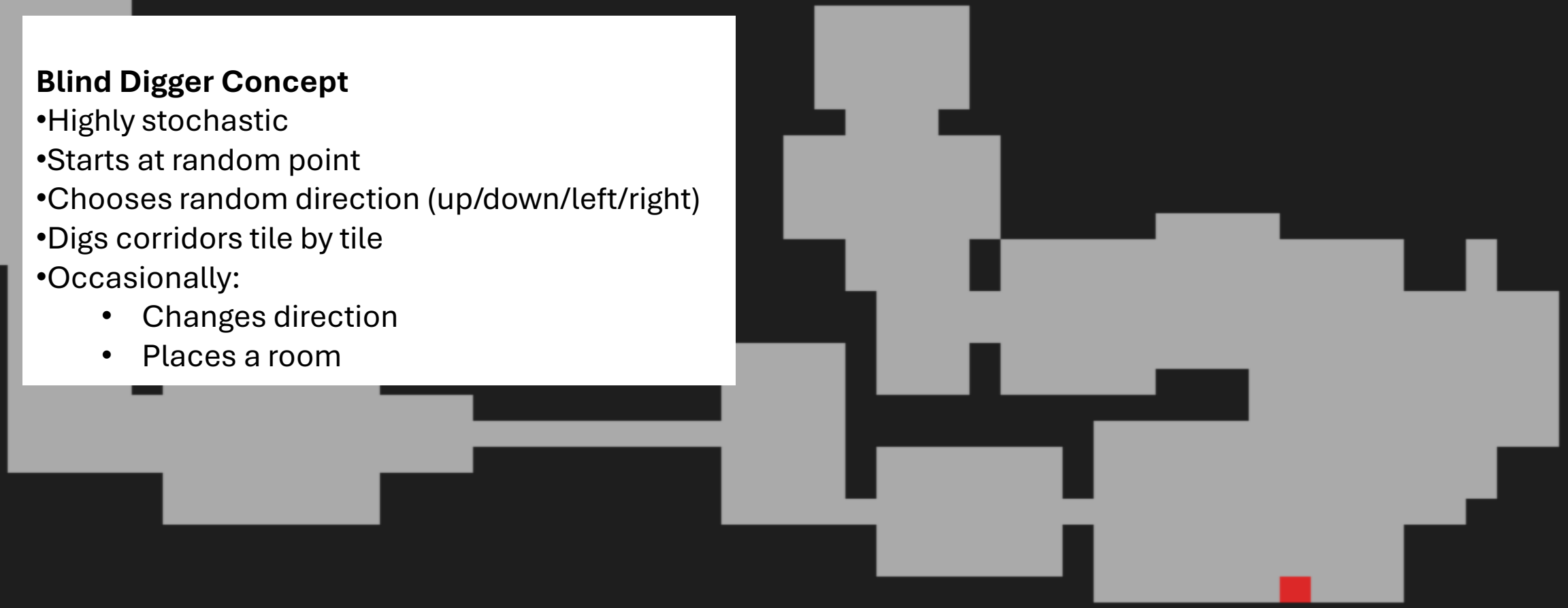
•Produces organic, chaotic layouts
•Less likely to create neatly organized dungeons

No guarantee of:
•No room overlap
•Full map coverage
•Balanced structure
•Infinite possible AI behaviors for diggers

Blind Digger Algorithm

Blind Digger Concept

- Highly stochastic
- Starts at random point
- Chooses random direction (up/down/left/right)
- Digs corridors tile by tile
- Occasionally:
    - Changes direction
    - Places a room



Blind Digger Algorithm

Dynamic Probabilities

- Initial:
  - PC (change direction) = 5%
  - PR (add room) = 5%
- Every tile:
  - If no direction change → PC += 5%
  - If no room added → PR += 5%
- After room placement → PR resets to 0%

Initialization:

1. Set `PC = 5%`, `PR = 5%`
2. Place agent at starting tile
3. Randomize direction (up, down, left, right)

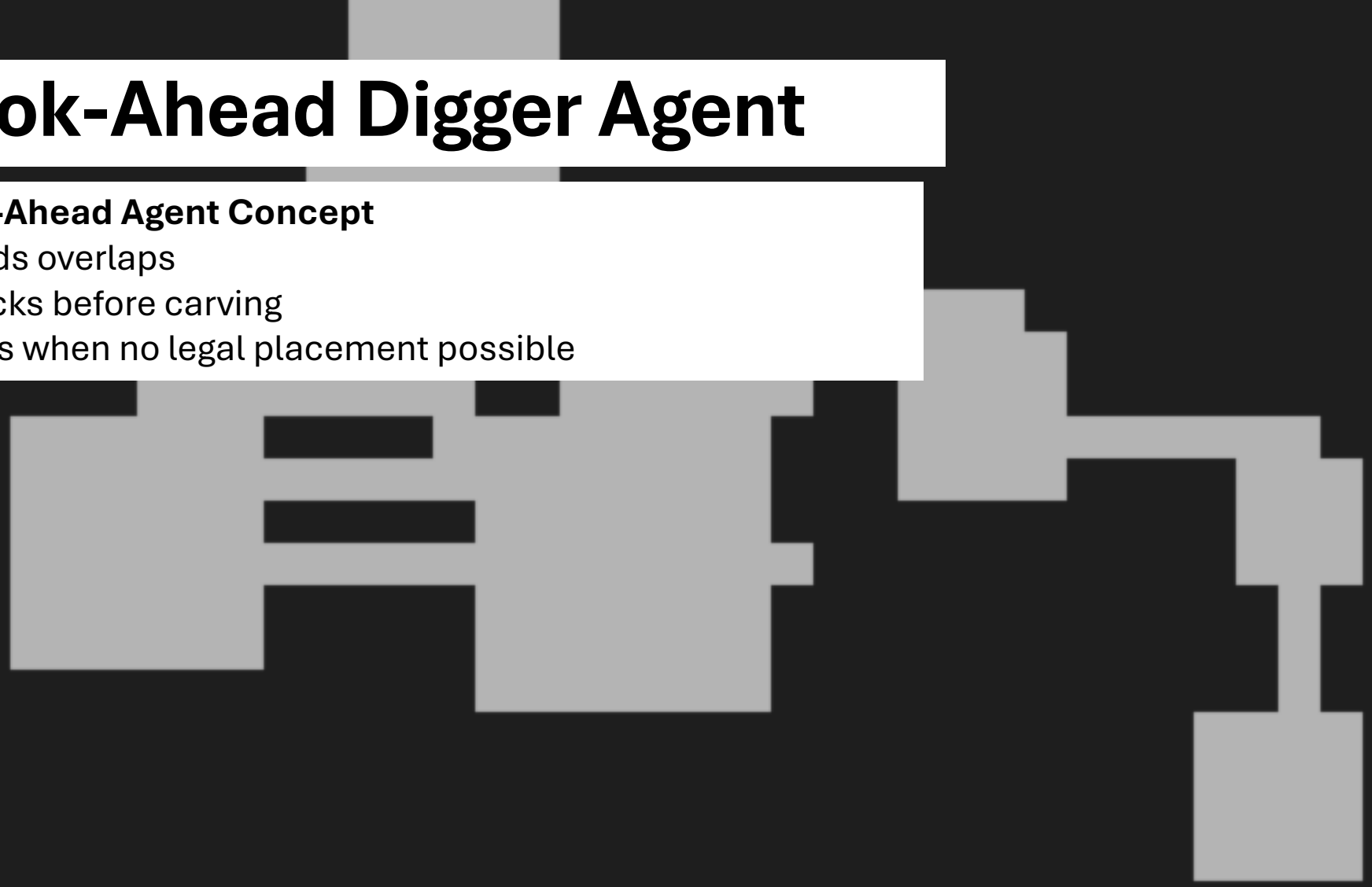
Loop:

1. Dig one tile → mark as corridor
2. Roll `NC` (0–100)
   1. If `NC < PC` → change direction, set `PC = 0`
   2. Else → `PC += 5%`
3. Roll `NR` (0–100)
   1. If `NR < PR` → place room (width, height = 3–7), `PR = 0`
   2. Else → `PR += 5%`

Look-Ahead Digger Agent

Look-Ahead Agent Concept

- Avoids overlaps
- Checks before carving
- Stops when no legal placement possible



Look-Ahead Digger Agent

Look-Ahead Algorithm (Stepwise)

Initialization:

1. Fill dungeon grid with walls
2. Place agent on starting tile → mark as floor

Room Placement Attempt:

1. Set `FR = 0` (room placed flag)
2. For each direction (up/down/left/right):
    1. For each room size (3–7 width × 3–7 height):
        1. Compute proposed room footprint
        2. If footprint does not intersect floor:
            1. Carve corridor from agent to room
            2. Carve room
            3. Move agent to a random perimeter tile of the room
            4. Set `FR = 1` and **break loops**

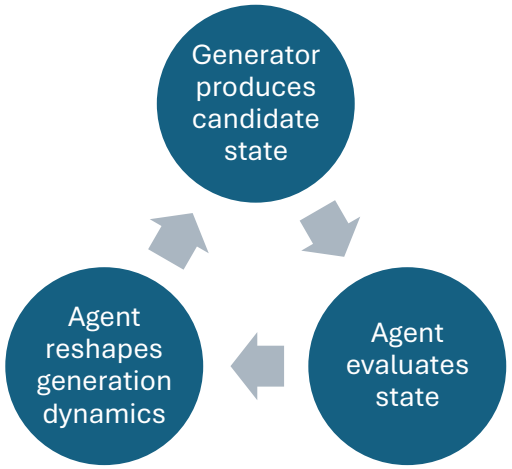
Corridor Placement (if no room):

1. If `FR == 0`:
    1. For each direction:
        1. For corridor length 3–7:
            1. Compute corridor footprint
            2. If corridor footprint does not intersect floor:
                1. Carve corridor
                2. Move agent to corridor end
                3. Set `FC = 1`
                4. Break loops

Termination:

Repeat until `FR == 0` and `FC == 0`

Pattern 2 — Agents as Evaluators



A - Agents **evaluate procedurally generated scenarios**.

In evolutionary strategies agents may act as a selection mechanism to access generated levels and mechanics

B –Agents **regulate in game experiences**, adapting content and gameplay.

It measures:

- Difficulty
- Emotional intensity
- Resource scarcity
- Spatial flow

Modifying parameters on the fly

Example: The AI Director

In Left 4 Dead:

- The AI Director monitors player stress.
- It tracks performance, damage taken, pacing.
- It dynamically adjusts enemy spawn timing and intensity.

Pattern 2 — Agents as Evaluators

Other Evaluation Roles

- Balancing agents (economy tuning)
- Difficulty regulators
- Aesthetic scoring agents
- Narrative tension monitors

Strengths

- Player-adaptive systems
- Controlled emergence
- Maintains pacing
- Supports personalization

Risks

- Overfitting to player behavior
- Oscillation (difficulty spikes)
- Loss of authorial structure
- Hidden systemic bias

Pattern 3 — Agents as Story Generators

Narrative is not written, it emerges from:

- Agent goals
- Resource competition
- Social relationships
- Environmental constraints

Story becomes a side effect of simulation.

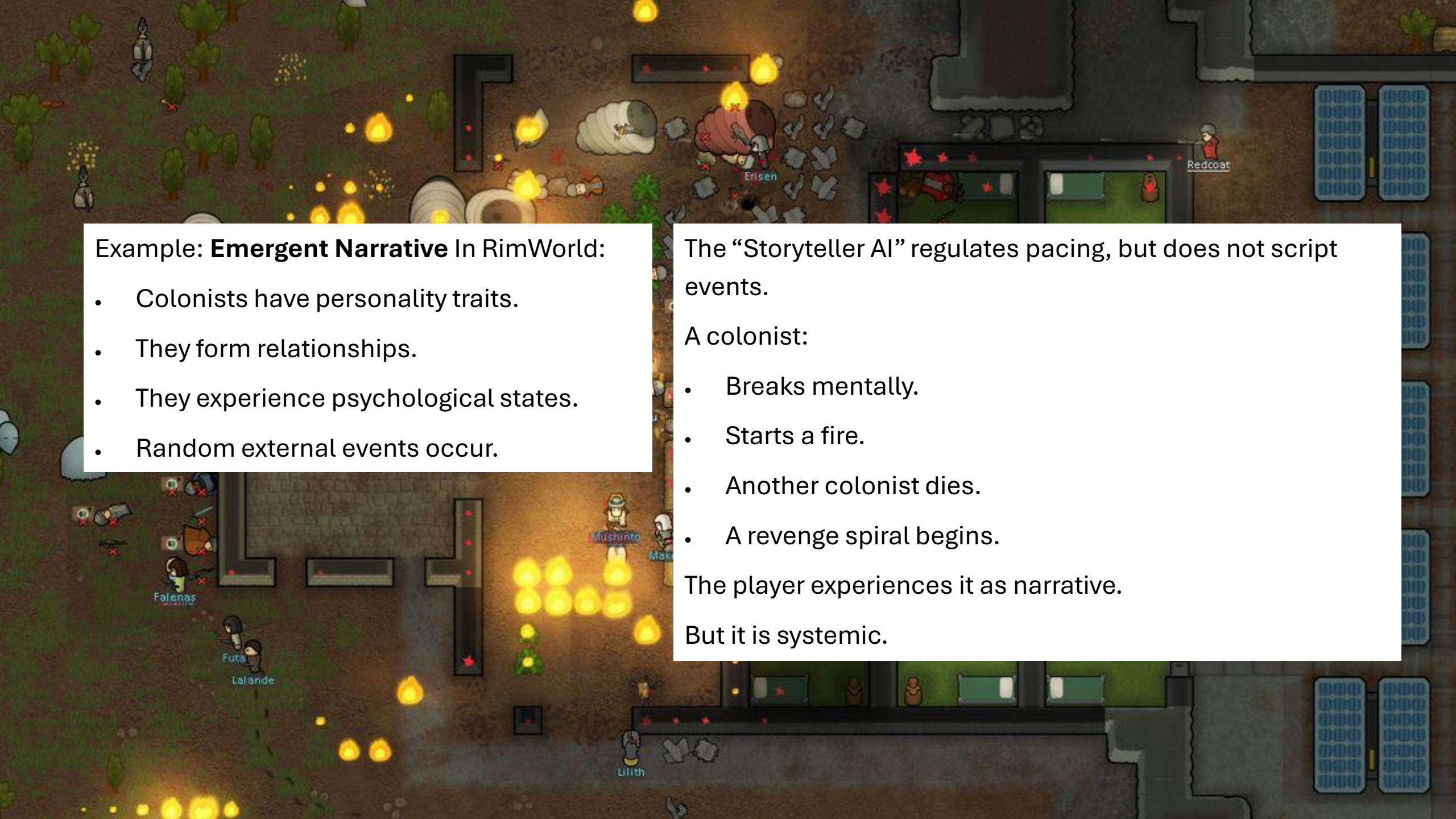
Conceptual Structure

Each agent has:

- **Desires**
- **Beliefs**
- Capabilities
- Relationships

Conflict arises when:

- Goals collide
- Resources are scarce
- Alliances shift
- Environmental pressures escalate



Example: **Emergent Narrative** In RimWorld:

- Colonists have personality traits.
- They form relationships.
- They experience psychological states.
- Random external events occur.

The “Storyteller AI” regulates pacing, but does not script events.

A colonist:

- Breaks mentally.
- Starts a fire.
- Another colonist dies.
- A revenge spiral begins.

The player experiences it as narrative.

But it is systemic.

Why use Agents for PCG?

- **Emergence**

Complex structures arise from simple rules.

- **Believability**

Content feels "grown" rather than placed.

- **Adaptivity**

Agents can respond to player behavior.

- **Multi-scale simulation**

Agents can operate at:

  - Micro scale (movement)
  - Meso scale (group behavior)
  - Macro scale (civilization growth)

- **Meta control**

Adapt gameplay

Core models of Agent-Based PCG

1. Swarm Systems

- Inspired by:
- Ant colonies
- Flocking birds
- Slime molds

Core models of Agent-Based PCG

2. Ecosystem Simulation

. Agents:

- Plants grow
- Herbivores eat plants
- Carnivores hunt herbivores

Terrain, migration paths, and population density emerge.

Used in simulation-heavy systems like *Dwarf Fortress*.

World history is generated through simulated agents acting over centuries.

Core models of Agent-Based PCG

3. Settlement Growth Agents

Instead of placing cities:

Agents:

- Choose resource-rich areas
- Expand near water
- Avoid harsh terrain
- Build roads toward other settlements

Road networks and city layouts emerge from expansion behavior.

Core models of Agent-Based PCG

4. Evolutionary Agents

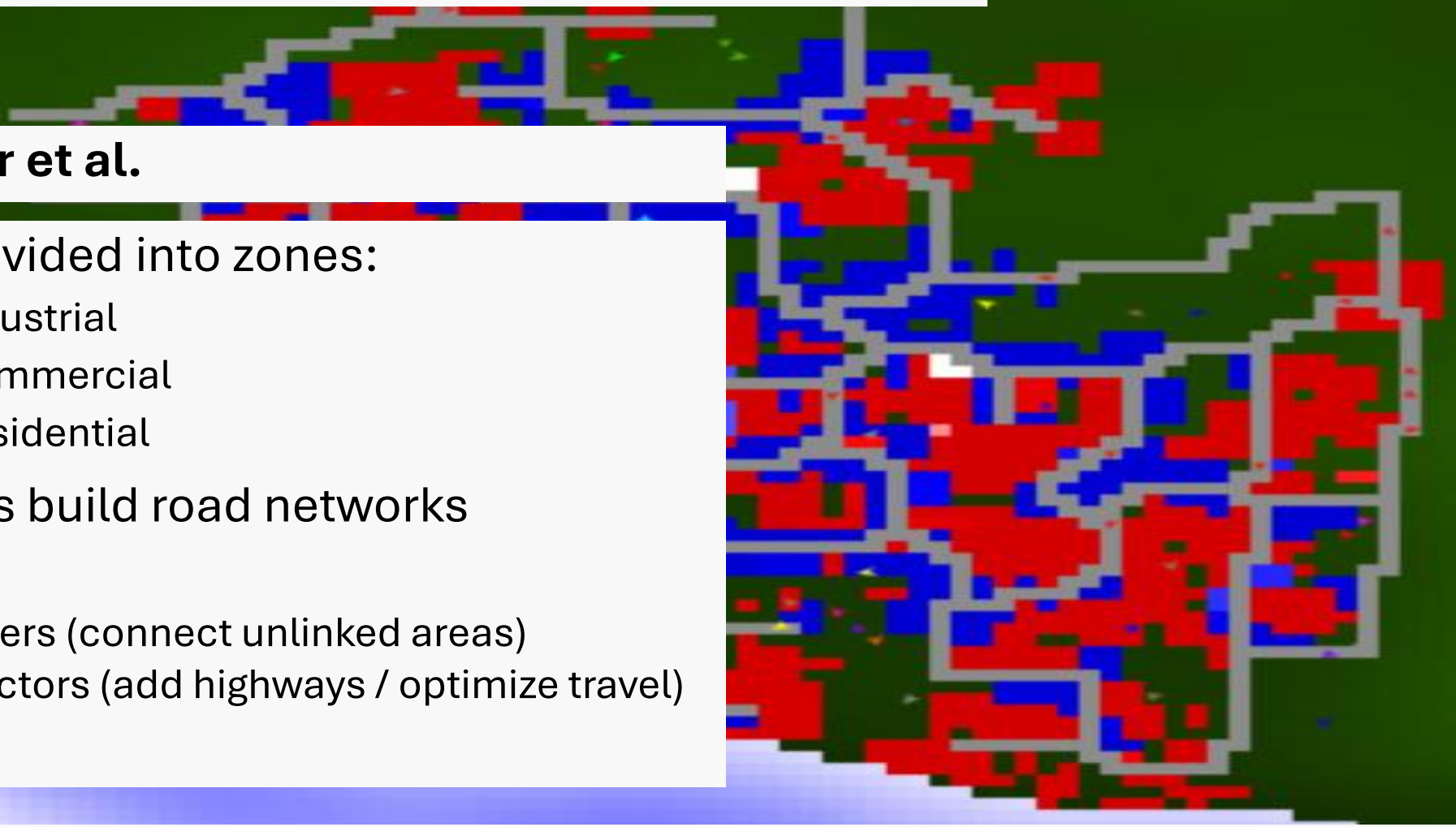
Agents:

- Compete
- Mutate
- Reproduce

Over generations, content evolves.

Example:
Weapon generation
Creature design
Strategy adaptation

Agent-Based **City Generation**



Lechner et al.

- City divided into zones:
  - Industrial
  - Commercial
  - Residential
- Agents build road networks

Types:
•Extenders (connect unlinked areas)
•Connectors (add highways / optimize travel)

Reference: Lechner, T., Watson, B. Wilensky, U. Procedural city modeling. In Proceedings of the 1st Midwestern Graphics Conference (2003)

Agent-Based **Terrain Generation**

Doran & Parberry

Three Phases
- Coastline shaping
- Smoothing
- Beach creation
- Mountain formation
- River carving

Agents have limited lifetimes.

Reference: Doran, J. Parberry, I.: Controlled procedural terrain generation using software agentes. IEEE Transactions on Computational Intelligence and AI in Games 2(2), 111-119 (2010)

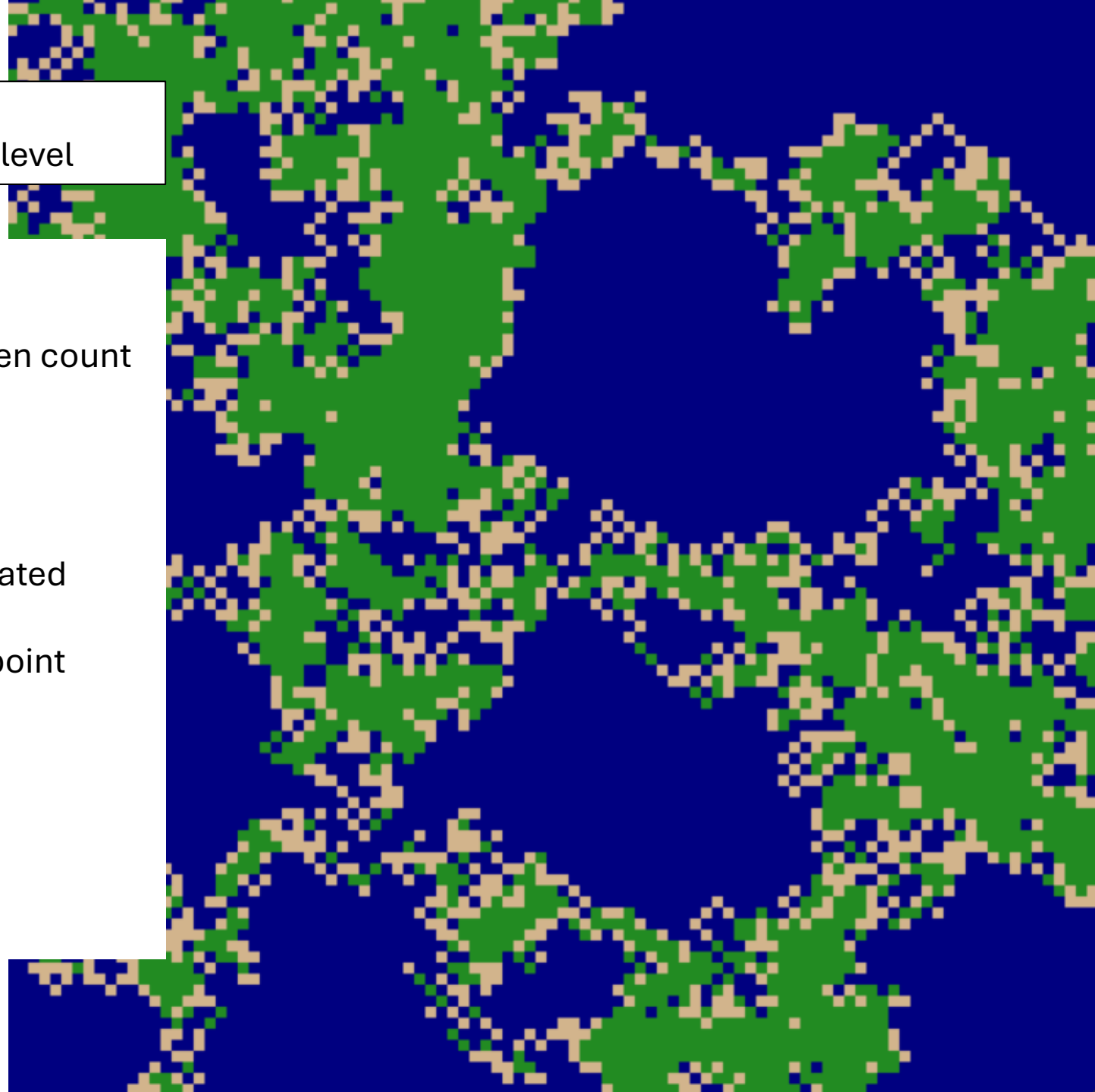
Coastline Agents
Purpose: Draw the outline of the terrain above sea level

Algorithm:

1. Initialize map below sea level
2. Place agent at map edge with a direction and token count
3. For each step:
    1. Generate 2 random points:
        1. Attractor → agent moves closer
        2. Repulsor → agent moves away
    2. Identify neighboring points that can be elevated above sea level
    3. Score points → elevate the highest scoring point
    4. Move agent to new point
    5. Decrement lifespan token
4. Repeat until agent's tokens run out

Notes:

- Multiple agents cover large maps
- Result: irregular, natural coastlines

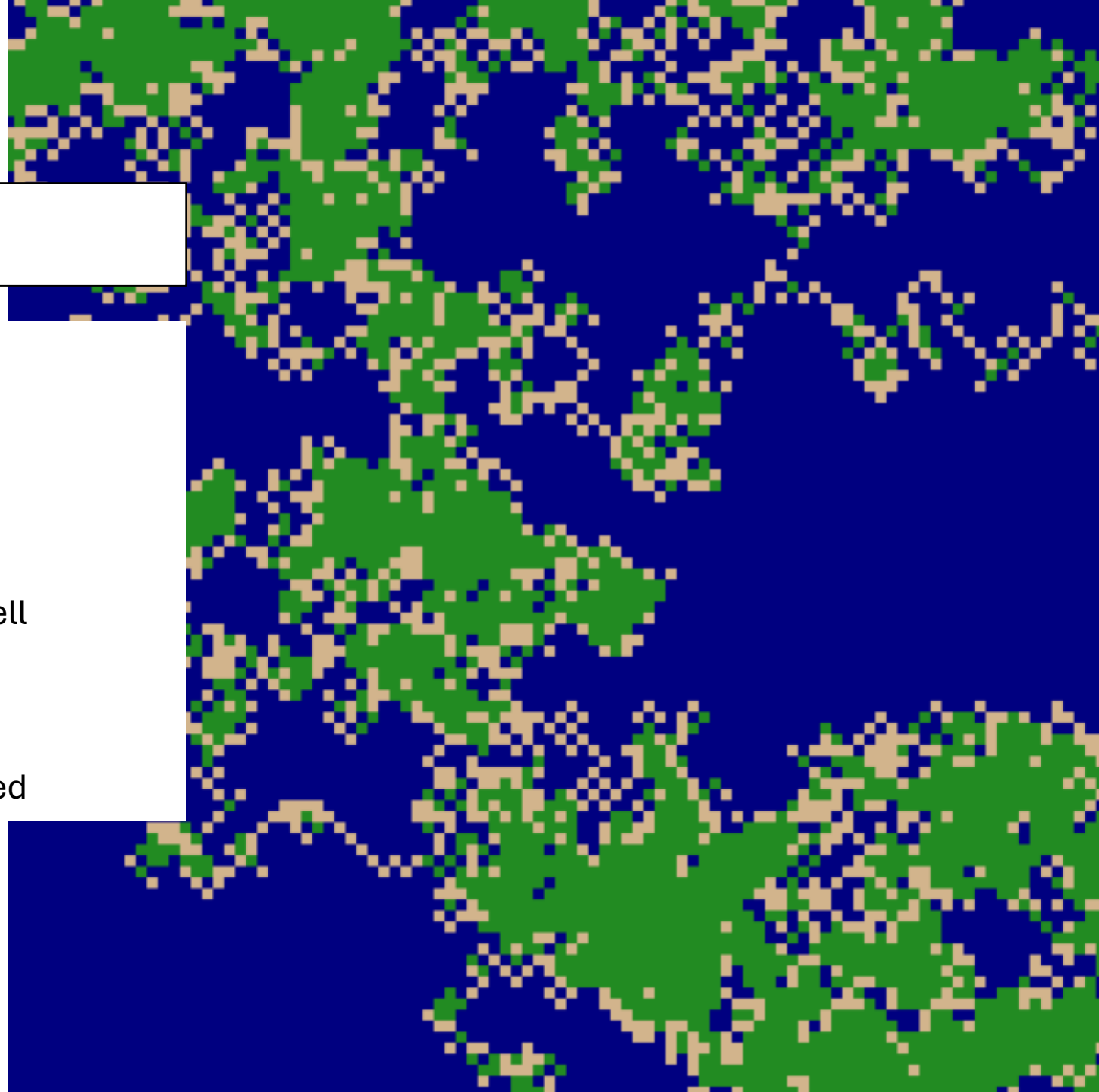


Smoothing Agents
Purpose: Eliminate abrupt elevation changes

Algorithm:

1. Place agents randomly on terrain
2. Each agent:
    1. Moves randomly to a neighboring cell
    2. Computes **average height** of neighbors:
        1. 4 orthogonal + 4 two steps away
        2. Weighted average including current cell
    3. Sets current cell height to average
    4. Decrement visits remaining
3. Repeat until all agents finish

Result: Terrain looks more natural, slopes softened



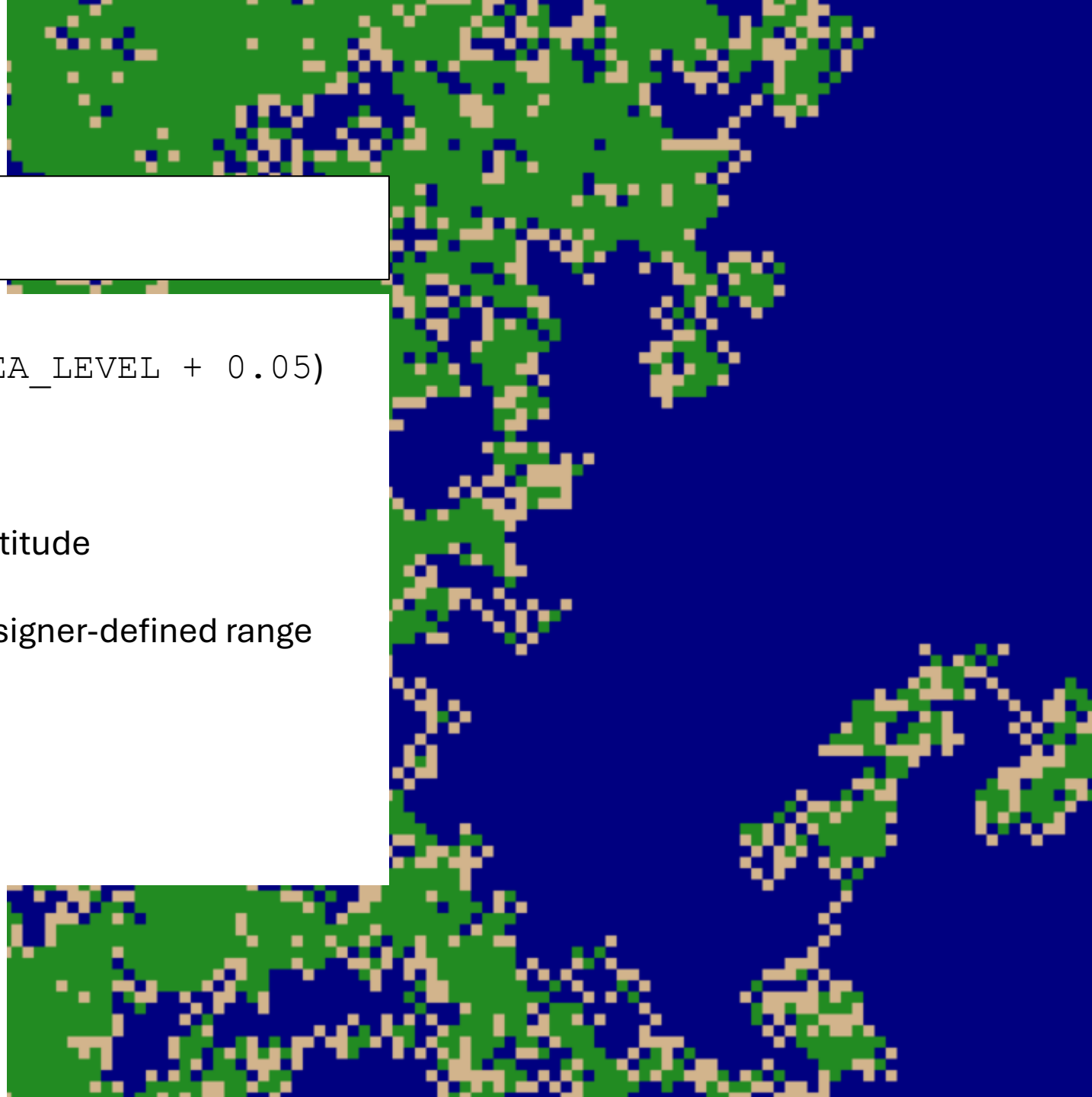
Beach Agents
Purpose: Flatten coastal areas into beaches

Algorithm:

1. Identify coastline tiles (`SEA_LEVEL ≤ h < SEA_LEVEL + 0.05`)
2. Place agents on coastline tiles
3. Each agent:
    1. Moves randomly along shore
    2. Checks tile height → must be below max altitude
    3. If tile within depth limit:
        1. Assign new height randomly within designer-defined range
    4. Decrement steps remaining
4. Repeat until all agents finish

Notes:

- Narrow height range → flat beaches
- Larger height range → bumpy/rocky beaches



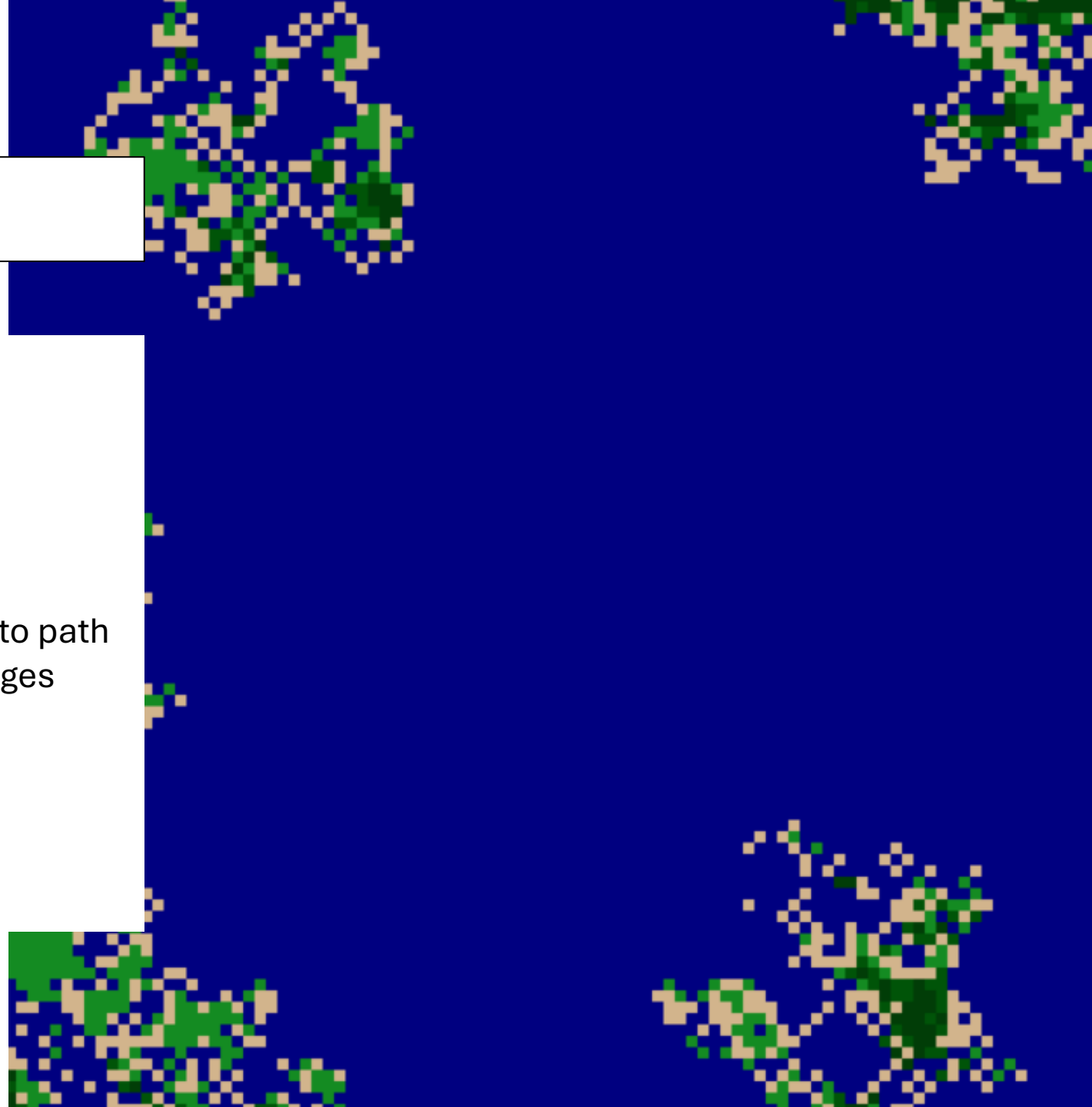
Mountain Agents
Purpose: Create ridges, peaks, and foothills

Algorithm:

1. Place agents randomly above threshold height
2. Each agent moves in a random direction
3. For each step:
    1. Elevate a V-shaped wedge perpendicular to movement
    2. Occasionally create **foothills** perpendicular to path
    3. Randomly turn ±45° occasionally → zigzag ridges
    4. Decrement steps remaining

Designer Controls:

- Slope drop rate, ridge width, maximum height
- Foothill frequency & length
- Number of agents & steps



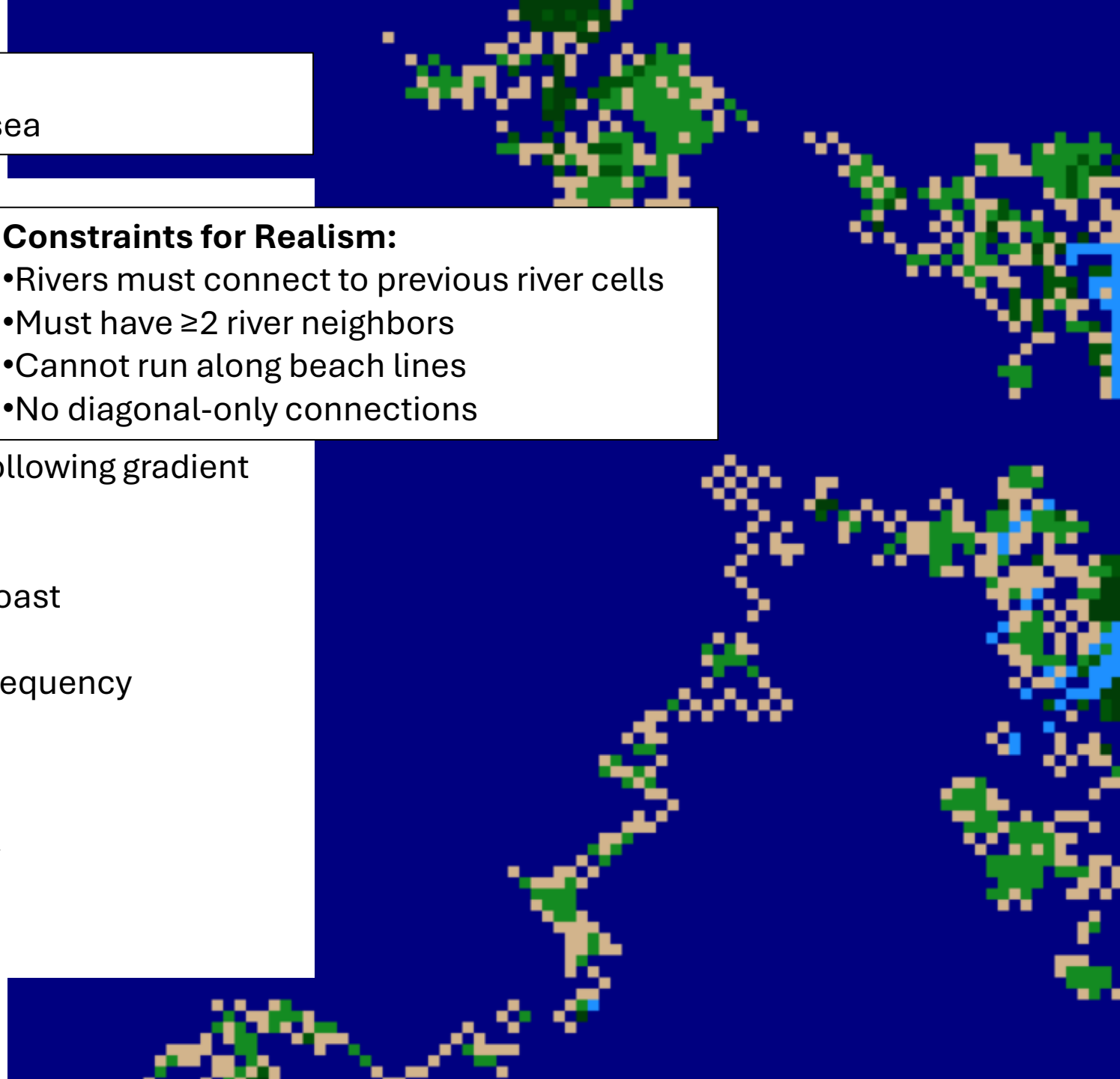
River Agents
Purpose: Carve rivers from mountains to the sea

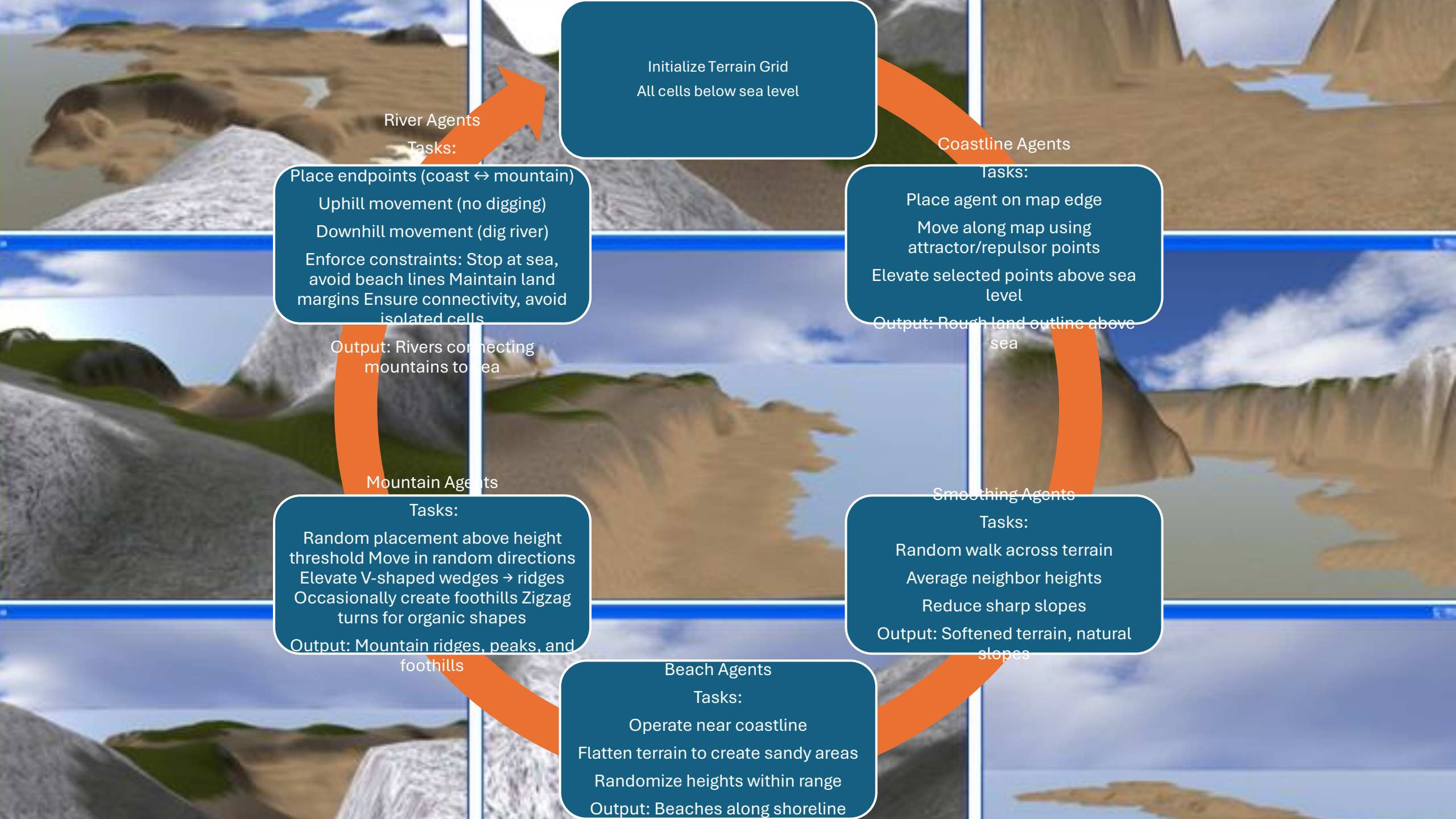
Algorithm:

1. Attempt to place river endpoints:
    1. Start: coastline tile
    2. End: mountain ridge tile
    3. Retry if river too short
2. **Uphill phase**:
    1. Agent moves from coast → mountain following gradient
    2. Does **not dig yet**
3. **Downhill phase**:
    1. Agent moves down from mountain → coast
    2. Carves river along path
    3. Adjust wedge width, slope, widening frequency
    4. Stop if:
        1. Sea is 1 tile away
        2. Land margin too narrow
        3. Digging would expose sea laterally
        4. Would create isolated river cells
4. Repeat until river agent finishes or fails

Constraints for Realism:

- Rivers must connect to previous river cells
- Must have ≥2 river neighbors
- Cannot run along beach lines
- No diagonal-only connections




Initialize Terrain Grid
All cells below sea level
Coastline Agents
Tasks:
Place agent on map edge
Move along map using attractor/repulsor points
Elevate selected points above sea level
Output: Rough land outline above sea
Smoothing Agents
Tasks:
Random walk across terrain
Average neighbor heights
Reduce sharp slopes
Output: Softened terrain, natural slopes
Beach Agents
Tasks:
Operate near coastline
Flatten terrain to create sandy areas
Randomize heights within range
Output: Beaches along shoreline
Mountain Agents
Tasks:
Random placement above height threshold Move in random directions Elevate V-shaped wedges → ridges Occasionally create foothills Zigzag turns for organic shapes
Output: Mountain ridges, peaks, and foothills
River Agents
Tasks:
Place endpoints (coast ↔ mountain)
Uphill movement (no digging)
Downhill movement (dig river)
Enforce constraints: Stop at sea, avoid beach lines Maintain land margins Ensure connectivity, avoid isolated cells
Output: Rivers connecting mountains to sea

Controllability Problem

Designers need:

- Artistic control
- Predictability
- Thematic coherence

Agent systems tend toward:
Unpredictability
Emergent chaos

The key research problem in PCG with agents:
How do we guide emergence without destroying it?

Solutions include:

- **Reward shaping**
- **Constraint systems**
- **Meta-agents**
- **Hybrid** rule-based + agent systems

Hybrid Systems (Advanced Design)

- Most sophisticated PCG systems combine all three:

Example structure:

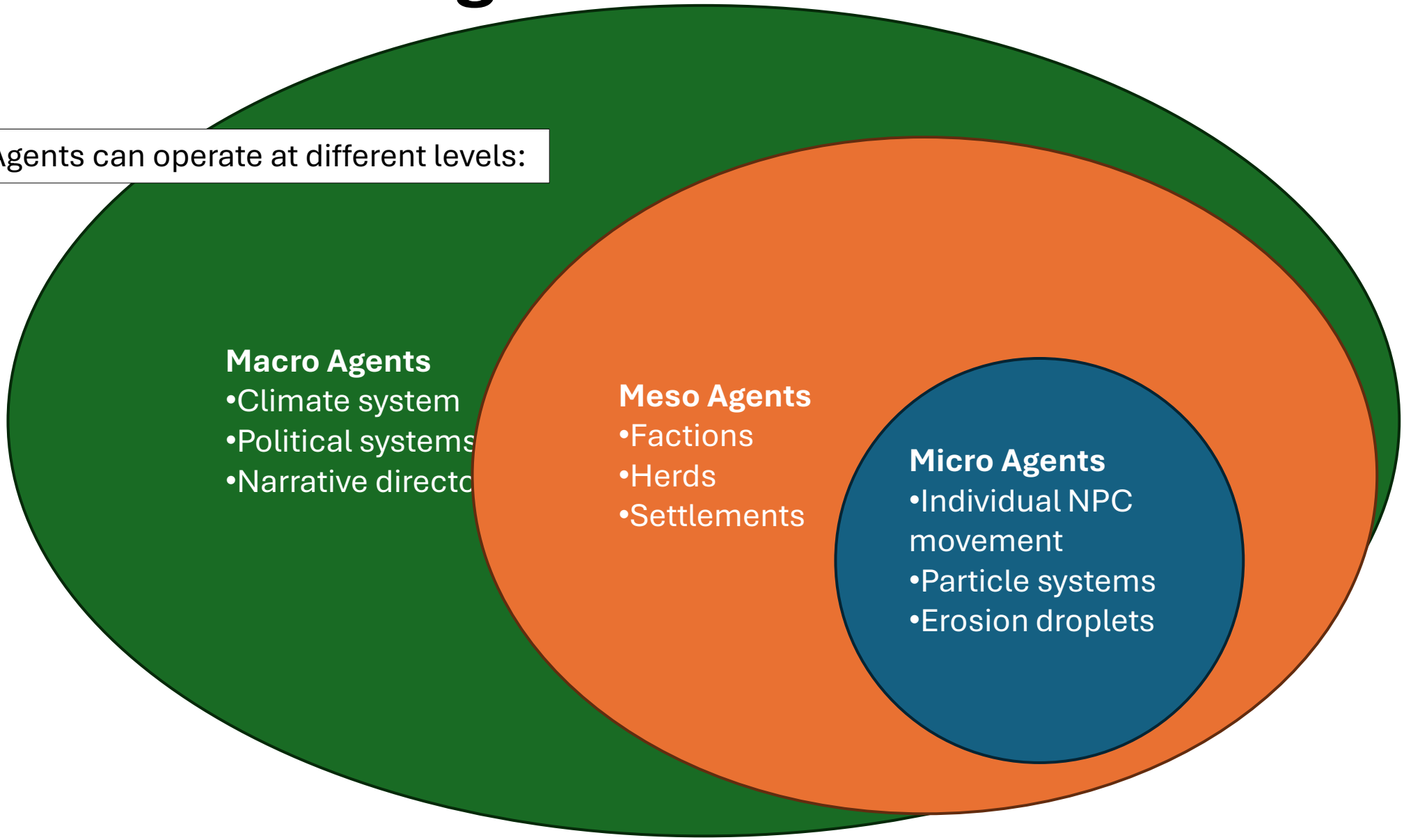
- **Sculptor** agents generate terrain.
- **Story agents** inhabit terrain.
- **Evaluator agent** regulates pacing.
- **Macro-agent** enforces thematic coherence.

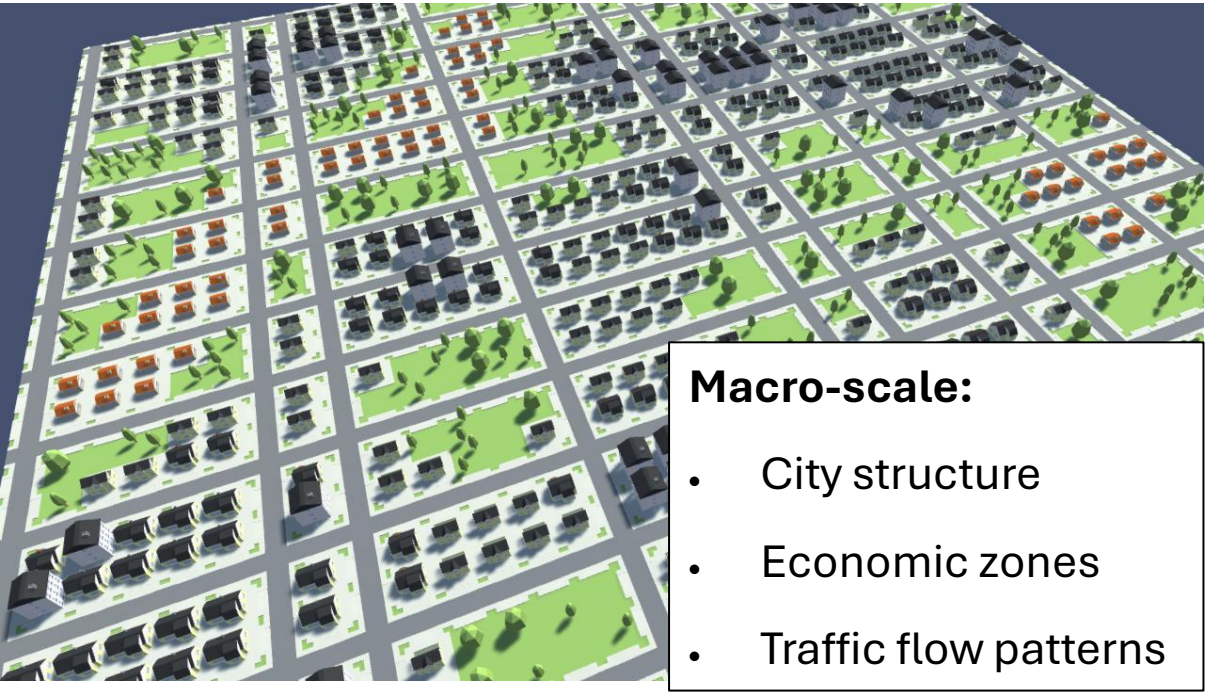
Procedural Content Generation

- Emergence vs Control
- Cellular Automata
- Agents
- **Multi-scale simulation problems**

Multi-Scale Agent PCG

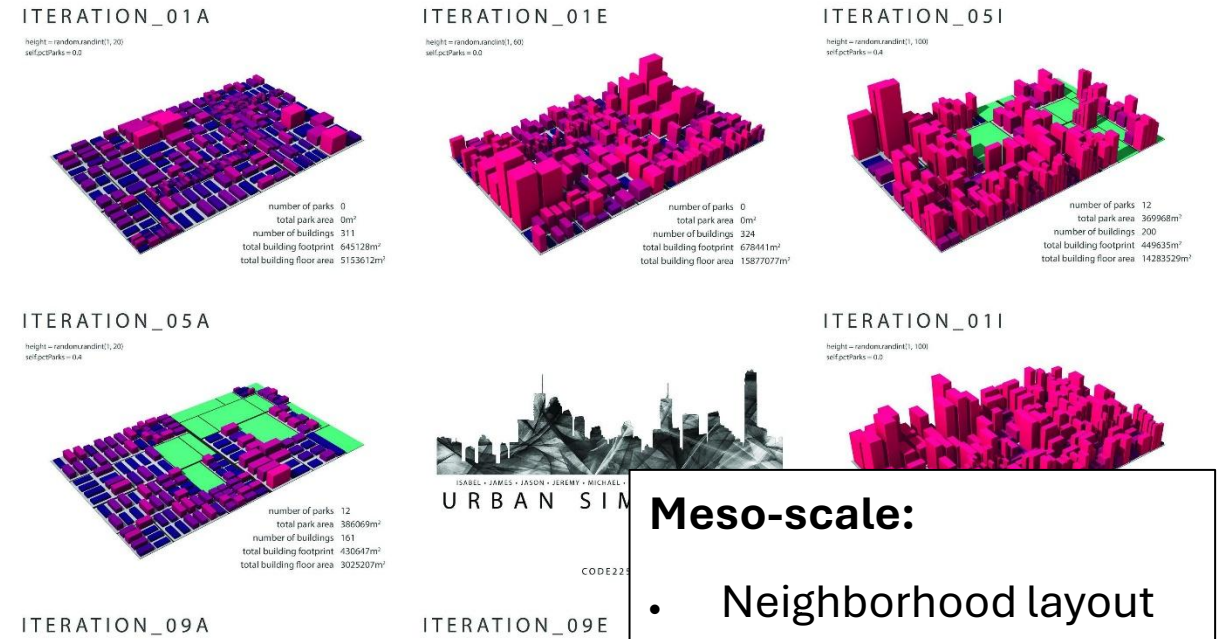
Agents can operate at different levels:





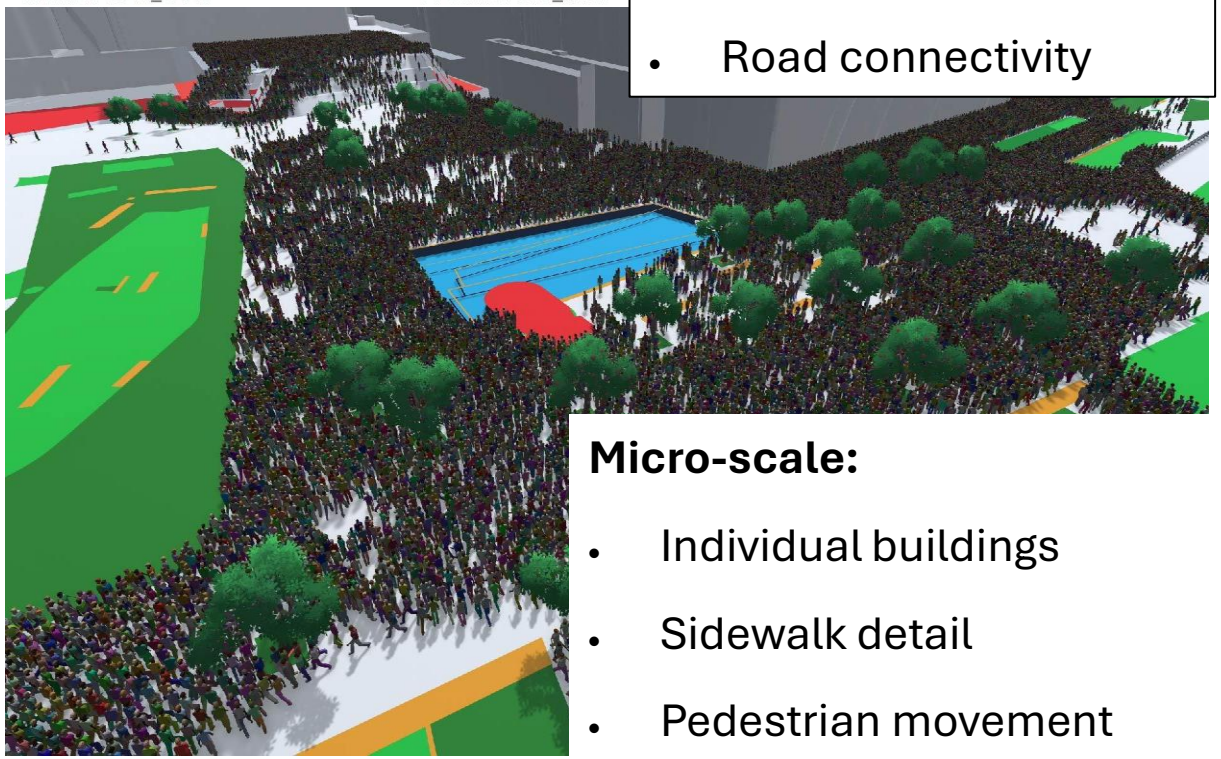
Macro-scale:

- City structure
- Economic zones
- Traffic flow patterns



Meso-scale:

- Neighborhood layout
- Road connectivity



Micro-scale:

- Individual buildings
- Sidewalk detail
- Pedestrian movement

Possible conflicts

- Individual agents → traffic jams may emerge.
- Zoning rules → might conflict with road generation.
- Road network → might ignore building density.

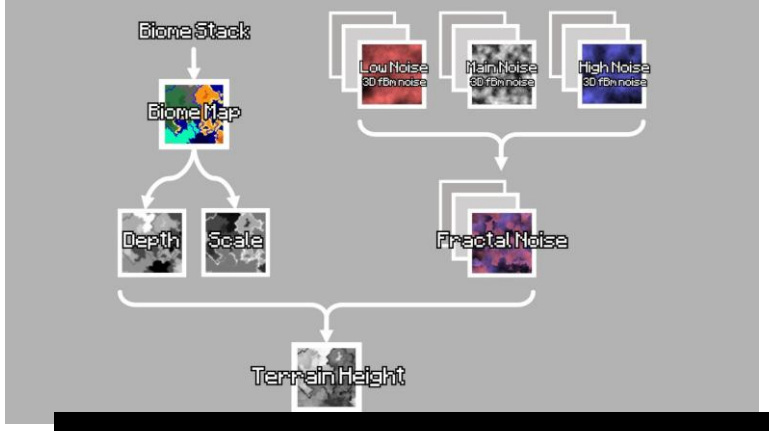
Multi-scale simulation problems in Procedural Content Generation (PCG)

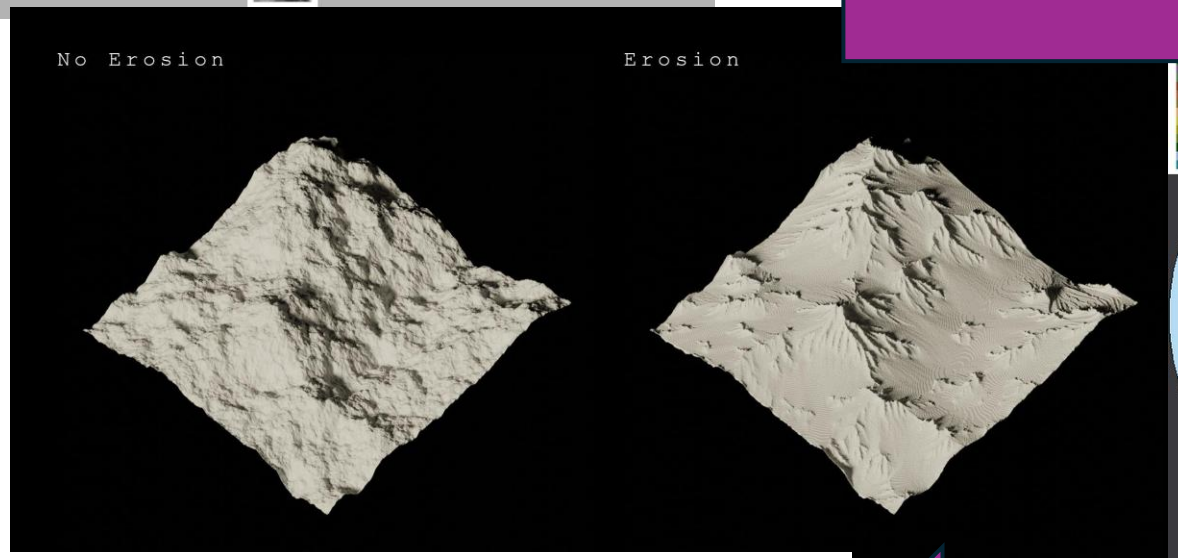
Multi-scale **consistency**

Multi-scale" means **different levels of size, detail, or abstraction happening simultaneously**.

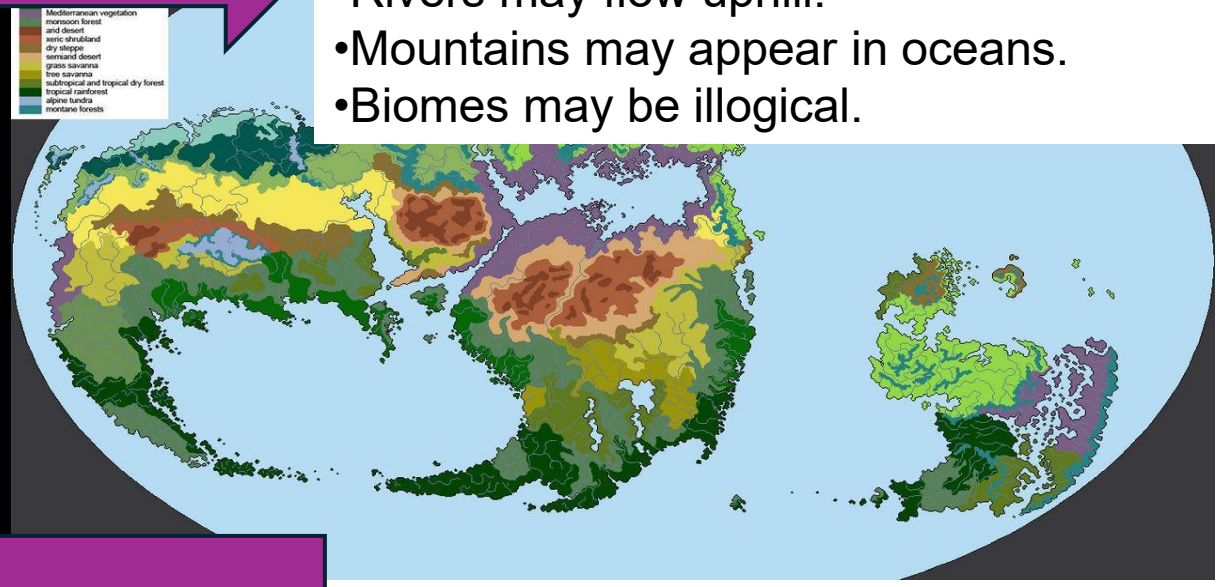
Strategy contradicts movement
Planner ignores physics
Local reactions destroy global intent
Emergent behavior becomes chaotic instead of believable

- **Behavior that works at one scale causes problems or inconsistencies at another scale.**
- Small-level interactions influence large-level structure — and vice versa.
- Procedural systems must simulate dependencies
- A multi-scale system must make sense at *all these levels*.





- Continents may look random blobs.
- Rivers may flow uphill.
- Mountains may appear in oceans.
- Biomes may be illogical.



- Noise might break river slopes.
- Erosion might invert intended drainage.
- Local variation might destroy the intended shape

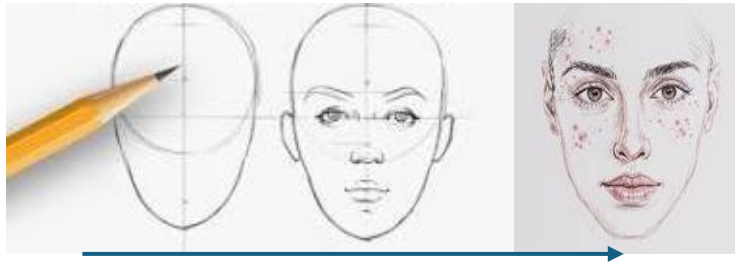
In AI Terms, it's a coordination problem between:

- Symbolic reasoning (plans, goals)
- Simulation systems (physics, movement)
- Reactive systems (steering, avoidance)
- Long-term models (economy, faction systems)
- Each scale influences the others.

Macro Layer
Faction politics
City economy
War simulation
Micro Layer
NPC daily routines
Pathfinding
Combat reactions

•Macro simulation says "city is starving"
•But NPCs still casually shop and eat
•NPC wants to go to market (high-level goal)
•Pathfinding fails (low-level behavior)

Why is this Hard ?



Emergence
Small rules produce unpredictable large patterns.
Feedback loops
Global structure influences local behavior.
Computational cost
Simulating every scale precisely is expensive.
Consistency constraints
Local detail must not contradict global logic

- High-level AI reasons abstractly.
- Low-level AI operates under physical constraints.
- Mid-level AI coordinates groups.
- Each layer uses different representations.
- They must all agree.